

Kubuntu 24.04 LTS System Configuration Guide

Production Workstation Environment Architecture & Constraints

1. Hardware Environment Specification

The system is deployed on a high-end desktop workstation with fixed, specific hardware topographies that govern operating system behaviors and virtual memory configurations.

Component	Specification	Administrative Consequences
Motherboard	ASUS TUF X299 Mark 1	Fixed PCIe lane topology: <code>PCIEX16_1</code> runs at x16 speed, <code>PCIEX16_3</code> runs at x8 speed, and <code>PCIEX4_1</code> is limited to x1 speed due to lane sharing. Intel RST couples the M.2_1 slot, <code>PCIEX4_2</code> , and SATA ports 5–8.
Processor	Intel Core i9-10900X (Cascade Lake-X)	10 Cores, 20 Threads, 48 PCIe lanes, dual AVX-512 FMA blocks, 165W TDP. CPU governor, turbo boost, and security mitigations require lower-level kernel configuration. AVX-512 optimization applies to local machine learning builds.
Memory	156.9 GiB DDR4-2933 (Quad-channel)	Provides substantial headroom for <code>tmpfs</code> and zram allocation, allowing high flexibility for virtual memory tuning (<code>vm.*</code> parameters).
Graphics	2x NVIDIA TITAN Xp Collectors Edition (Pascal architecture, GP102)	Hardware is bound to legacy driver constraints. This acts as a primary technical limitation for graphics and compute configurations.
Audio	Native Instruments Komplete Audio 6 (USB)	Handled via PipeWire with custom device prioritization and channel routing.
Network	Intel I219-V + Intel I211	Network interface naming managed via <code>udev</code> rules, feeding directly into NetworkManager profiles. One interface remains active while the other is disabled.
Cooling	NZXT Kraken AIO Liquid Cooler	Pump and fan telemetry read through <code>lm-sensors</code> kernel modules where exposed.
Storage	4-Drive Storage Array	System OS is installed on <code>/dev/sdd</code> . File system diagnostics are required for handling potential read-only or locking states.

2. Operating System Baseline & Desktop Environments

Stock Configuration

- **Distribution:** Kubuntu 24.04 LTS (Noble Numbat).
- **Desktop Components:** KDE Plasma 5.27.11, KWin 5.27.11, KDE Frameworks 5.x, Qt 5.
- **Default Display Protocol:** X11 session. Wayland is available only as an experimental option via the `plasma-workspace-wayland` package and lacks official support on the stock Plasma 5.27 base.
- **Kernel:** Stock GA kernel 6.8 (subject to updates via the Hardware Enablement (HWE) point releases).

Plasma 6 Upgrade Path

- **Availability:** KDE Plasma 6, Qt 6, and KDE Frameworks 6 can be pulled into this system exclusively by enabling the Kubuntu Backports PPA repository.

3. NVIDIA Driver & Pascal Architecture Constraints

The NVIDIA TITAN Xp graphics cards are built on the Pascal architecture. NVIDIA has designated the R580 driver branch as the final release series to support Maxwell, Pascal, and Volta architectures. The R590 branch completely drops support for these legacy cards.

Critical Operational Rule: The graphics driver must be frozen on the 580 legacy branch (terminal version: `580.105.08`). Upgrades to 590 or newer must be explicitly blocked using APT pinning.

Configuration Requirements

1. **Driver Version Locking:** Upgrades to 590 or newer must be blocked natively via package preferences.
2. **Kernel Module Selection:** Only the proprietary closed-source kernel modules are compatible (`nvidia-driver-580`). The open-source NVIDIA kernel modules (`-open`) require a Turing architecture or newer GPU equipped with a Graphics Processing Cluster (GSP) engine.
3. **Compute Framework Ceiling:** Machine learning frameworks are limited to a CUDA 12.x toolkit ceiling. CUDA 13 and subsequent versions drop Pascal support. All Python virtual environments must use packages compiled against CUDA 12 compatible libraries.
4. **Display Protocol Interactions:** While dual Pascal GPUs can run a Wayland session under Plasma 6, the X11 display protocol remains the more reliable, lower-risk environment for this hardware generation. On the stock Plasma 5.27 desktop environment, X11 is the only stable choice.
5. **Multi-GPU Compute Configuration:** Scalable Link Interface (SLI) is not utilized for compute workloads. The two GP102 GPUs operate as independent CUDA devices. Management is performed using `nvidia-persistenced` alongside per-device power and clock configurations via `nvidia-smi`.
6. **Kernel Compatibility and Security:** Post-EOL, these GPUs receive only minor security patches and basic compatibility adjustments for newer Linux kernels. Every HWE kernel upgrade must be manually verified against the locked 580 DKMS build before deployment.

4. Core System Configuration Layers

Bootloader and Initial Ramdisk

- **Tooling:** UEFI settings, GRUB (`/etc/default/grub`), `initramfs-tools` , `mokutil` , and `intel-microcode` .
- **Configurations:** Manages boot priority, Resizable BAR, Above-4G decoding for multi-GPU addressing, VT-d, and IOMMU setups. Secure Boot compliance requires signing the closed NVIDIA kernel modules using Machine Owner Keys (MOK). Core kernel parameters include `nvidia-drm.modeset=1` .

Kernel and Hardware Parameters

- **Tooling:** `/etc/modprobe.d/` , `/etc/sysctl.d/` , `nvidia-smi` , `intel_pstate` , `lm-sensors` , and `udev` .
- **Configurations:** Disables the open-source `nouveau` driver, configures NVIDIA kernel module options, optimizes virtual memory settings, manages CPU scaling governors/turbo frequencies, sets GPU power caps, maps multi-GPU output routing, and defines predictable network interface names via `udev` .

Initialization and Core Services

- **Tooling:** Systemd service units, systemd targets (`graphical.target`), `systemd-logind` , `systemd-journald` , and `systemd-timesyncd` .
- **Configurations:** Manages target initialization, system journal persistence limits, and hardware clock synchronization. Power, sleep, and lid events are explicitly assigned to either `logind.conf` or the desktop environment's power manager to avoid duplicate event triggering. System-wide locales are managed through `localectl` .

Storage and Filesystem Management

- **Tooling:** `/etc/fstab` , LVM, LUKS, `crypttab` , `mdadm` , and zram utility scripts.
- **Configurations:** Mount optimization (e.g., `noatime`), SSD trim operations via `fstrim.timer` , disk array configuration, and zram swapping parameters paired with virtual memory tuning (`vm.swappiness`).

Read-Only Filesystem Troubleshooting: If a volume falls into a read-only state or encounters an unmount lock, use the following verification sequence before assuming filesystem corruption:

```
findmnt
mount | grep ' ro '
ls -ld /path/to/target
lsblk -o NAME,RO
blockdev --getro /dev/sdX
lsattr /path/to/target
```

Identity, Authentication, and System Security

- **Tooling:** `/etc/pam.d/` , `/etc/security/limits.conf` , Polkit rules (`/etc/polkit-1/rules.d/`), `sudoers.d` , `ufw` , and AppArmor.

- **Configurations:** User group permissions (`video` , `render` , `sudo`), PAM authentication stacking, resource limitations, Polkit desktop escalation rules, network firewall policies, and application containment profiles.

5. Package Management & Software Deployment

APT Configuration & Repository Pinning

To enforce the legacy driver requirements and manage optional desktop environment upgrades:

- **Kubuntu Backports PPA:** Acts as the toggle between the stock configuration (Plasma 5.27) and the updated desktop stack (Plasma 6).
- **NVIDIA 580 Branch Pinning:** Run the following command to lock current versions:

```
sudo apt-mark hold nvidia-driver-580 nvidia-dkms-580 'libnvidia-*580'
```

Create an explicit repository configuration file at `/etc/apt/preferences.d/nvidia-580-pin` with the following content to block version 590 or higher:

```
Package: nvidia-driver-* nvidia-dkms-* libnvidia-*  
Pin: version 580*  
Pin-Priority: 1001
```

- Ensure that `unattended-upgrades` is configured to exclude these held packages.

Alternative Package Environments

- **Flatpak & Snap:** Manages sandboxed application installations. Flatpak global overrides are used for handling specific application permissions.
- **Python Runtime Policies:** Adheres to PEP 668 (externally managed environment restrictions in Ubuntu 24.04). Global use of `--break-system-packages` is restricted. Python environments must use local virtual environments or the `uv` toolchain matching the host's CUDA 12.x capabilities.

6. Display, Graphics, and Window Management

Display Setup

- **Configuration Utilities:** KScreen configurations located in `~/.local/share/kscreen/` or controlled via `kscreen-doctor`.
- **Scaling Rules:** On the stock X11 platform, global scaling combined with font DPI manual overriding provides the best results. Under a Plasma 6 Wayland configuration, native per-display fractional scaling is used.
- **Advanced Color Features:** High Dynamic Range (HDR) and advanced color spaces require the Plasma 6 Wayland environment and are unavailable on the stock Plasma 5.27 base.

Window Management (KWin)

- **Configuration Storage:** Managed through `kwinrc` and window-specific rules in `kwinrulesrc`.

- **Controls:** Handles window layout behaviors, window rules, tiling configurations, custom desktop visual effects, edge activation actions, and compositing backends. Configurations can be reloaded without a full logout using D-Bus:

```
qdbus org.kde.KWin /KWin reconfigure
```

7. Desktop Portals & Permission Layers

The desktop portal layer handles access controls between applications and the host system. While primarily acting as a lightweight translation interface under X11, it is critical under Wayland for managing security boundaries, screen streaming, and desktop integration.

Core Architecture

- **Main Service:** `xdg-desktop-portal` acts as the primary system request router.
- **KDE Backend:** `xdg-desktop-portal-kde` provides native KDE Qt dialog boxes and permission enforcement.
- **Fallback Backend:** `xdg-desktop-portal-gtk` offers fallback interfaces for properties not natively covered by the KDE backend.

Key Integration Interfaces

- **Screencasting and Remote Desktop:** Routes video streams through PipeWire. Screen recording utilities (e.g., OBS Studio) and desktop sharing (KRDP) depend on this interface working in tandem with the NVIDIA legacy driver.
- **File Chooser Integration:** Delivers native KDE open and save dialog windows to containerized flatpak or snap applications.
- **Input Capture:** Manages requests from external or simulated input utility applications requiring global keyboard shortcuts or mouse manipulation under Wayland.
- **System Settings Synchronization:** Exposes the host color schemes and theme configurations to GTK-based and sandboxed applications.

To troubleshoot issues such as failing screen shares or incorrect file dialog styles, inspect the portal system logs:

```
journalctl --user -u xdg-desktop-portal -u xdg-desktop-portal-kde
```

8. Display Manager & Session Initialization

- **Display Manager Configuration:** SDDM configurations are organized within `/etc/sddm.conf.d/`. These files control the greeter appearance theme, default session selection (X11 vs Wayland), automatic logins, and boot-time NumLock behavior.
- **Session Environment Initialization:** Environment scripts are parsed from `~/.config/plasma-workspace/env/` and `~/.config/environment.d/`. Autostart applications use standard desktop launchers inside `~/.config/autostart/`.

9. Plasma Workspace Shell & Themes

- **Layout Configuration:** Layout states, panel arrangements, and active widgets are stored inside `~/.config/plasma-org.kde.plasma.desktop-appletsrc`.
- **Widget Management:** Custom desktop widgets are deployed using `kpackagetool5` (under stock configurations) or `kpackagetool6` (under Plasma 6 Backports).
- **Workspace Theme Control:** Applies system themes, styles, application color maps (`kdeglobals`), window decorations, icons, fonts, and cursors. Application behavior configurations are bridged to GTK applications via `kde-gtk-config`.

10. Device & Connectivity Control

Hardware Peripheral Profiles

- **Input Configuration:** Handles keyboard mappings, key repeat delays, pointer behaviors via `libinput`, and drawing tablet workspace calibration.
- **Audio Routing:** Managed using the PipeWire audio system graph, controlling default hardware sound output priorities and per-application volume controls.
- **Hardware Connections:** Manages Bluetooth pairings, automated mounting behaviors for removable external storage devices, printing systems via CUPS, and USB hardware access restrictions.

Network Services (User Level)

- **Configuration Tools:** NetworkManager profiles managing wireless connections, static or dynamic IP addressing, DNS overrides, and VPN plugin authentication.
- **Remote Connectivity:** Direct network file sharing through Samba (`kio-smb`) and native remote desktop servers (VNC or KRDP).

11. Data Indexing, App Config, & Web Integrations

Data Indexing and Search

- **File Indexing:** Baloo file search engine configurations (`balooctl`). It may be disabled if high storage I/O constraints occur.
- **Desktop Search Launcher:** KRunner search behavior preferences, file extension associations, and application execution defaults (`mimeapps.list`).

Application Specific Configurations

- Core application configuration files are isolated inside `~/.config/` (such as `konsolerc`, `dolphinrc`, `katerc`, and `spectaclerc`).
- **Credential Vault:** KWallet wallet storage systems and automated PAM-based credential unlocking parameters.

Integrating Local Web Services as System Menu Entries

To integrate a locally hosted web service into the native system application menus:

1. **Create the Desktop Launcher File:** Write a custom desktop configuration file to `~/.local/share/applications/local-service.desktop`:

```
[Desktop Entry]
Type=Application
Name=Local Web Application
Comment=Local web service interface
Exec=xdg-open http://localhost:8080/
Icon=web-browser
Categories=Network;Development;
Keywords=local;development;dashboard;
StartupNotify=false
```

2. **Update the Desktop Database:** Register the new file with the system application index:

```
update-desktop-database ~/.local/share/applications
```

3. **Configure Web Instance Isolation (Optional):**** To open the local address inside a dedicated borderless window independent from standard browser sessions:

- **Chromium/Brave:** Change the launch command to:

```
Exec=chromium-browser --app=http://localhost:8080/ --user-data-dir=%h/.local/share/webapps/local-service
```

- **Taskbar Identity Mapping:** Define a specific `StartupWMClass=` string in the desktop entry file so the window manager groups the application correctly, matching any specific window behavior rules.

12. Configuration Backup, Restoration, & Maintenance

Configuration Tooling

Manual configuration scripts or desktop maintenance shortcuts rely on the following system utilities:

- **Stock Architecture (Plasma 5):** Uses `kreadconfig5`, `kwriteconfig5`, `kcmshell15`, and `kbuildsycoca5`.
- **Backports Architecture (Plasma 6):**** Uses `kreadconfig6`, `kwriteconfig6`, `kcmshell16`, and `kbuildsycoca6`.

Backup Locations for System Restoration

To back up or restore this workstation configuration, track the following specific directories and configuration file paths:

- **User Configuration Files:**

- `~/.config/*rc` (Includes `kwinrc` , `kwinrulesrc` , `kglobalshortcutsrc` , `kdeglobals`)
- `~/.config/plasma-org.kde.plasma.desktop-appletsrc`
- `~/.config/plasma-workspace/`
- `~/.local/share/plasma/`
- `~/.local/share/kscreen/`
- `~/.local/share/applications/` (Custom launcher configurations)

- **System Global Configurations:**

- `/etc/apt/preferences.d/nvidia-580-pin`
- `/etc/modprobe.d/` (NVIDIA module parameters)
- `/etc/sddm.conf.d/` (Display manager settings)
- `/etc/default/grub` (Boot configuration options)